King's College London
Department of Informatics
Dr. Mohammad Abdulaziz

Computer-Assisted Theorem Proving: 7CCSACTP

Exercise Sheet 6

Important Note: Please bring a laptop with Isabelle/HOL installed to the Large Group Tutorial (LGT). Performing these proofs in Isabelle will be an integral part of the tutorial.

Exercise 6.1 Merge Sort

Consider the following version of merge sort: it splits a list into two halves, sorts them recursively and merges the results. Unlike the version in the lecture, it treats the empty and the one-element list by separate equations.

(a) Elements. Prove that *msort* preserves the elements of its input: the output has the same multiset of elements as the input.

```
fun merge :: 'a::linorder list ⇒ 'a list ⇒ 'a list where
  merge [] ys = ys
| merge xs [] = xs
| merge (x#xs) (y#ys) = (if x ≤ y then x # merge xs (y#ys) else y # merge (x#xs) ys)

fun msort :: 'a::linorder list ⇒ 'a list where
  msort [] = []
| msort [x] = [x]
| msort xs = (let n = length xs div 2 in merge (msort (take n xs)) (msort (drop n xs)))
```

Prove that merging two lists preserves their combined multiset of elements.

```
lemma mset_merge: mset (merge xs ys) = mset xs + mset ys
  sorry
```

Prove that merge sort preserves the multiset of elements from the original list.

```
lemma mset_msort: mset (msort xs) = mset xs
  sorry
```

(b) Sortedness. Part (a) showed that *msort* keeps the elements of its input. The other half of its correctness is that its output is sorted. Prove the following lemmas, first pen-and-paper, then in Isabelle.

```
lemma set_merge: set (merge xs ys) = set xs ∪ set ys
```

```
  sorry

lemma sorted_merge: sorted (merge xs ys) ⟷ sorted xs ∧ sorted ys
  sorry

lemma sorted_msort: sorted (msort xs)
  sorry
```

Hints: 1. All three lemmas are proved by computation induction.

2. *sorted* $(x \# ys)$ requires x to be at most every element of ys; this is why *sorted_merge* needs *set_merge*.

Exercise 6.2 Running Time

Consider list append, the naive reverse function, which appends each element at the end, and the tail-recursive reverse function from Week 3:

```
fun app :: 'a list ⇒ 'a list ⇒ 'a list where
  app [] ys = ys
| app (x # xs) ys = x # app xs ys

fun reverse :: 'a list ⇒ 'a list where
  reverse [] = []
| reverse (x # xs) = app (reverse xs) [x]

fun itrev :: 'a list ⇒ 'a list ⇒ 'a list where
  itrev [] acc = acc
| itrev (x # xs) acc = itrev xs (x # acc)
```

(a) Using the counting rules from the lecture, with all constants reduced to 1 (count the time functions of the defined functions that are called, plus 1 for the call itself), define the time functions *t_app*, *t_reverse* and *t_itrev* in Isabelle.

(b) Prove, first pen-and-paper and then in Isabelle, that *reverse* takes quadratic and *itrev* linear time: *t_reverse xs* $\leq$ *(length xs + 1)^2* and *t_itrev xs acc = length xs + 1*.

Hints: 1. First prove *t_app xs ys = length xs + 1* and *length* (*reverse xs*) = *length xs*.

2. For *t_itrev*, think about which variable the induction hypothesis must be generalised over.

3. See Section 1.2 (Running Time) of the FDS book.

(c) Comparisons. For sorting algorithms that compare list elements, a common cost measure is the number of comparisons between list elements, instead of all function calls. The following functions count the comparisons (the tests $x \leq y$) that *merge* and *msort* from Exercise 6.1 make:

```
fun c_merge :: 'a::linorder list ⇒ 'a list ⇒ nat where
  c_merge [] ys = 0
| c_merge xs [] = 0
| c_merge (x#xs) (y#ys) = 1 + (if x ≤ y then c_merge xs (y#ys) else c_merge (x#xs) ys)

fun c_msort :: 'a::linorder list ⇒ nat where
  c_msort [] = 0
| c_msort [x] = 0
| c_msort xs = (let n = length xs div 2; ys = take n xs; zs = drop n xs
                in c_msort ys + c_msort zs + c_merge (msort ys) (msort zs))
```

Prove, first pen-and-paper and then in Isabelle, that sorting a list of length $2\hat{}k$ takes at most $k * 2\hat{}k$ comparisons.

```
lemma c_merge_ub: c_merge xs ys ≤ length xs + length ys
  sorry

lemma length_msort: length (msort xs) = length xs
  sorry

lemma c_msort_le: length xs = 2 ^ k ⟹ c_msort xs ≤ k * 2 ^ k
  sorry
```

Hints: 1. *c_merge_ub* and *length_msort* are proved by computation induction; *length_msort* needs *length (merge xs ys) = length xs + length ys*.

2. Prove *c_msort_le* by induction on *k*, generalising over *xs*. In the step case, both halves of *xs* have length $2\hat{}k$.

3. The number of comparisons of merge sort is analysed in Chapter 2 of the FDS book.

Exercise 6.3 Bottom-Up Merge Sort

Bottom-up merge sort does not split the list. It starts from singleton lists and repeatedly merges adjacent pairs of lists until a single list is left.

```
fun merge_adj :: ('a::linorder) list list ⇒ 'a list list where
  merge_adj [] = []
| merge_adj [xs] = [xs]
| merge_adj (xs # ys # zss) = merge xs ys # merge_adj zss
```

Isabelle needs the following lemma to show that *merge_all* below terminates.

```
lemma length_merge_adjacent[simp]: length (merge_adj xss) = (length xss + 1) div 2
```

```
  by (induction xss rule: merge_adj.induct) auto

fun merge_all :: ('a::linorder) list list ⇒ 'a list where
  merge_all [] = []
| merge_all [xs] = xs
| merge_all xss = merge_all (merge_adj xss)

definition msort_bu :: ('a::linorder) list ⇒ 'a list where
  msort_bu xs = merge_all (map (λx. [x]) xs)
```

(a) Correctness. Prove that *msort_bu* preserves the elements of its input and that its output is sorted, first pen-and-paper, then in Isabelle.

```
lemma mset_msort_bu: mset (msort_bu xs) = mset xs
  sorry

lemma sorted_msort_bu: sorted (msort_bu xs)
  sorry
```

Hints: 1. Neither lemma follows by induction on *xs* directly. First prove lemmas about *merge_adj* and *merge_all* on an arbitrary list of lists *xss*: that they preserve *mset* (*concat xss*), and that they produce sorted lists if all lists in *xss* are sorted.

2. Prove those lemmas by computation induction, with *merge_adj.induct* and *merge_all.induct*.

(b) Running time. We count comparisons as in Exercise 6.2(c):

```
fun c_merge_adj :: ('a::linorder) list list ⇒ nat where
  c_merge_adj [] = 0
| c_merge_adj [xs] = 0
| c_merge_adj (xs # ys # zss) = c_merge xs ys + c_merge_adj zss

fun c_merge_all :: ('a::linorder) list list ⇒ nat where
  c_merge_all [] = 0
| c_merge_all [xs] = 0
| c_merge_all xss = c_merge_adj xss + c_merge_all (merge_adj xss)

definition c_msort_bu :: ('a::linorder) list ⇒ nat where
  c_msort_bu xs = c_merge_all (map (λx. [x]) xs)
```

Prove that sorting a list of length *2^k* takes at most $k * 2\hat{}k$ comparisons.

```
lemma c_msort_bu: length xs = 2 ^ k ⟹ c_msort_bu xs ≤ k * 2 ^ k
  sorry
```

Hints: 1. Use *c_merge_ub* from Exercise 6.2(c).

2. If all lists in *xss* have length *m*, then *c_merge_adj xss* ≤ *m* ∗ *length xss*; if moreover *length xss* is even, all lists in *merge_adj xss* have length *2* ∗ *m*.

3. Generalise the statement to *c_merge_all*: if all lists in *xss* have length m and *length xss* $= 2\hat{}k$, then *c_merge_all xss* $\leq m * k * 2\hat{}k$. Prove it by computation induction (*c_merge_all.induct*), generalising over k and m.